

FarmEasy

*Project Report Submitted in Partial
fulfilment of the requirement for the
award of Degree of*

MASTER OF COMPUTER APPLICATION (MCA)

Submitted by

Priyanshu Baral

Reg No: 2314514533

Under the guidance of

Mrs. Lipika Sharma

Guide Reg No:



MANIPAL UNIVERSITY JAIPUR (MUJ)

CENTRE FOR DISTANCE & ONLINE EDUCATION

July & 2025

BONAFIDE CERTIFICATE

Certified that this project report titled FarmEasy is the bonafide work of “Priyanshu Baral” who carried out the project work under my supervision in the partial fulfilment of the requirements for the award of the Master of Computer Application (MCA) degree.

A handwritten signature in black ink, appearing to read 'Lipika', with a stylized flourish at the end.

SIGNATURE

Name of the Guide (Lipika Sharma)

Guide Registration Number

DECLARATION BY THE LEARNER

I, Priyanshu Baral bearing Reg. No 2314514533 hereby declare that this project report entitled Priyanshu Baral has been prepared by me towards the partial fulfilment of the requirement for the award of the Master of Computer Application (MCA) Degree under the guidance of Mrs. Lipika Sharma.

I also declare that this project report is my original work and has not been previously submitted for the award of any Degree, Diploma, Fellowship, or other similar titles.



Place: Delhi

Priyanshu Baral

Date: 20-07-2025

Reg. No. 2314514533

TABLE OF CONTENTS

List of Figures.....	III
List of Tables.....	V
List of Standards	VI
Abbreviations.....	VII
Abstract.....	VIII
CHAPTER 1. INTRODUCTION.....	1
1.1. Identification of Client/ Need/ Relevant Contemporary issue.....	1
1.2. Identification of Problem.....	2
1.3. Identification of Tasks.....	6
1.4. Timeline.....	7
1.5. Organization of the Report	8
CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY	9
2.1. Timeline of the reported problem.....	9
2.2. Existing solutions	10
2.3. Bibliometric analysis.....	12
2.4. Review Summary	17
2.5. Problem Definition	20
2.6. Goals/Objectives.....	21
CHAPTER 3. DESIGN FLOW/PROCESS.....	24
3.1. Evaluation & Selection of Specifications/Features.....	24
3.2. Design Constraints.....	26
3.3. Analysis of Features and finalization subject to constraints.....	29
3.4. Design Flow.....	31
3.5. Design selection.....	33
3.6. Implementation plan/methodology.....	35

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION	39
4.1. Implementation of solution.....	39
CHAPTER 5. CONCLUSION AND FUTURE WORK.....	49
5.1. Conclusion.....	49
5.2. Future work	52
REFERENCES.....	55
APPENDIX	58
1. Plagiarism Report.....	58
2. Design Checklist	58
USER MANUAL	59

List of Figures

Image 1.....	21
Image 2.....	21
Image 3.....	25
Image 4.....	26
Image 5.....	32
Image 6.....	32
Image 7.....	33
Image 8.....	33
Image 9.....	34
Image 10.....	34
Image 11.....	35
Image 12.....	35
Image 13.....	36
Image 14.....	36
Image 15.....	37
Image 16.....	37
Image 17.....	37
Image 18.....	37
Image 19.....	38
Image 20.....	38
Image 21.....	39
Image 22.....	39
Image 23.....	39
Image 24.....	40
Image 25.....	41

Image 26.....	41
Image 27.....	41
Image 28.....	42
Image 29.....	42
Image 30.....	43
Image 31.....	44
Image 32.....	44
Image 33.....	45
Image 34.....	45
Image 35.....	46
Image 36.....	47
Image 37.....	49
Image 38.....	49
Image 39.....	50
Image 40.....	51
Image 41.....	58
Image 42.....	59
Image 43.....	59
Image 44.....	59

List of Tables

Table 1.....	19
Table 2.....	22
Table 3.....	41
Table 4.....	42
Table 5.....	45
Table 6.....	47
Table 7.....	48
Table 8.....	49
Table 9.....	52
Table 10.....	52
Table 11.....	53
Table 12.....	53
Table 13.....	54
Table 14.....	56
Table 15.....	59
Table 16.....	62

Introduction / Objective:

Accessing finance is still a big problem for many farmers – they usually don't have the right support or time to go through those long loan processes. Most small farmers can't deal with the complicated paper work or online forms, and lot of them don't even get a chance cause of lack of digital skills or government delays. That's where FarmEasy comes in – it's made to make the whole loan thing simpler, quicker, and more farmer friendly.

The motive behind this project is to make a portal that actually connects farmers directly with the loan options available – from banks, gov schemes, or private ones. No middlemans, no too much red tape. The system checks who's eligible, helps with login securely, and also shows how the loan is processing in real time. All of this saves time and avoid going here and there with bunch of documents.

FarmEasy is built with a strong backend part (that handles all the heavy stuff) and easy front UI so even less techy farmers can use it. Also some bank APIs are added to make things smooth. There's features like comparing different loans, applying straight from the portal, and checking if it's approved or not. We also tested this out and it works well in demo setups.

In short, we trying to help farmers get loans without all the headache. If this works on large scale, it could really boost farming, support rural peoples, and also make sure food supply keeps strong in future.

System Analysis

1.1. Identification of Client /Need / Relevant Contemporary issue

Getting access to finance still remains one of the biggest & ongoing struggles for farmers in India—especially the small & marginal ones who actually make up like 85-86% of the whole agri workforce (as per some NABARD report). Even though they're basically the backbone of the country's economy and play a huge role in food security, most of these farmers end up stuck in loan cycles or poverty. The main issue is, they just can't get loans from proper channels that easily.

There's too many problems like—complicated forms, long waits, no nearby banks in villages, and also not knowing how to go about the financial stuff properly. And if that wasn't enough, they're often asked for heavy documents or land papers as collatorals which many of them don't even have. So, instead, lots of farmers just go to local moneylenders who charge insane interest—somewhere around 40 to 60% yearly—which just makes their financial condition more worst and stops them from investing in good seeds, new machines or water systems and better farming.

That's exactly why FarmEasy was created. It's basically a smart online portal built mainly for farmers, so they don't have to run pillar to post for loans. It connects them directly with a range of loan providers—like normal govt. banks, private banks, small loan companies (MFIs) and schemes from government side too. The site's been made real simple & easy-to-understand, so even farmers who ain't too used to tech can figure it out without getting stuck or lost. Not everybody's got time or skills to deal with confusing websites or apps, so we just kept it straight forward as possible.

It cuts down on all that extra red-tape stuff and saves lotta time. Farmers don't gotta run around asking 10 people what to do. They can check themselves which loan suits them or what schemes they fit into. No more guesswork or getting fooled by middlemans – now they can use FarmEasy and make smarter, more confident choices when it comes to money and their farms. Key features of AgriLend include:

1. Quick Loan Checks without the Wait: FarmEasy checks if a farmer is eligible for loans instantly—by looking at stuff like verified land papers, past farming data, credit history and all that. This saves tons of time and cuts out the guessing part that usually

happens with normal loan stuff. You don't have to wait for days just to know if you're eligible.

2. Know About Govt Subsidies & Schemes (Finally!): There's so many gov schemes that farmers don't even know exist. FarmEasy puts it all in one place—like PM Kisan Credit Card (PM-KCC), Infrastructure Fund, Fasal Bima Yojna and state-based subsidies too. And it's written in simple words, so people actually understand what they're reading, not like the usual govt brochure types.
3. Everything Online – No More Running to Banks: Farmers can apply for loans straight from their phones or computer. No need to go to the bank branch again and again wasting money and time. With e-KYC, uploading docs online, and tracking the whole loan process in real time, the whole thing becomes way less of a headache. You just apply, wait a bit, and see how it's going—without leaving your village.
4. It Teaches Too, Not Just Gives Loans: FarmEasy's not just another loan website. It also teaches farmers how to deal with money better. There's videos, short guides, FAQs and webinars (yes, even in local languages) on things like how to repay loans, planning your spending, making a credit score, or what interest even means. Stuff that banks never bother to explain properly.

Recent surveys show more than 65% of farmers face big problems when trying to get loans. Things like too many rules, not having land papers or other collaterals, and waiting forever for money to come. RBI reports also say that just 30% of small farmers are actually getting help from real banks. The rest still borrow from moneylenders who charge crazy interest and trap them in debt.

And when farmers can't get loans, they can't buy better seeds, fertilizers, machines or irrigation stuff. That means lower crops, less money, and the same struggle every season. It also hurts the bigger picture—like rural growth, food supply, and the overall economy.

FarmEasy is trying to actually fix the system using tech and smart ideas. By working closely with loan providers and banks and giving farmers personal help, we're trying to:

- Take off the financial pressure on farmer families.
- Help them build credit and become more stable.
- Make farming more profitable, not just survivable.
- Support rural development and financial inclusion in real way.

FarmEasy believes that no farmer should be stopped from growing just cause they didn't get a loan in time. With better access to credit, that's actually made for them, FarmEasy is helping shape a future where Indian farmers don't just survive—they succeed.

1.2. Identification of Problem

Getting loans through formal channels is still one of the biggest problems for Indian farmers today—specially for the small and marginal ones, who actually make up nearly 86% of all the country's farming workforce (as per NABARD). These farmers are the real backbone of India's food system and rural economy, but when it comes to getting money for seeds, tools, fertilisers, water supply or storing crops, they run into endless troubles.

One big reason behind this is that many rural farmers still don't have the kind of financial knowledge needed to deal with banks. Most of them don't fully understand how formal credit works, what documents are needed, what a credit history means, or how loan repayment works in the long run. That's why most of them either feel too nervous to even step into a bank, or they go ahead and try but just get rejected in the end. And honestly, that only adds more pressure and disappointment.

Collateral's another big mess. Many farmers just own a tiny piece of land, and half the time the paperwork isn't even clear—or it's split up between relatives, which makes it hard to use as security. So even if they really need money and could pay it back after the harvest, they don't qualify because they don't have the “right” documents or land ownership proof. Banks just reject the loan without looking at their actual situation.

Then there's the process itself—it's honestly just too long and complicated. So many steps, piles of paperwork, visiting far-away banks multiple times, and waiting for weeks or months to even hear back. Many farmers can't afford to take so much time away from their farms. Plus, not all villages have proper banking set-ups either. The whole system just drains them out.

Even when a farmer somehow manages to apply, the delay in actually getting the money is another headache. Agriculture is all about timing—missing the sowing season by a week can ruin everything. But the funds from approved loans often arrive late, and that's enough to spoil the entire season, putting farmers further into loss and struggle.

Because of all this, many farmers end up borrowing from local moneylenders. Yeah, it's fast and easy, but farmers gotta pay a heavy price for it. These local lenders take crazy high

interest — sometimes like 40 to 50 percent or even more every year — and trap farmers deep in debt. Instead of using their money to fix their farms or live better, farmers just keep paying off that interest again and again.

Another thing is that many farmers just don't know enough about government schemes that could actually help them. There are things like PM-KCC, Agri Infra Fund, state subsidies and so on—but awareness is super low. Even if they've heard the names, they don't know how to apply or think they won't be eligible. Some are also afraid they'll get stuck in hidden conditions or recovery trouble.

This whole gap between what banks and the system offer, and what farmers actually experience, is a serious issue. There's not enough on-ground communication, no farmer-friendly outreach, and very little digital support. So even when loans are “available,” they don't really reach the ones who need them the most. That's why farming stays stuck—low output, no modern tech, no risk coverage, and no growth.

Farmers who can't access timely and affordable credit also can't invest in good quality seeds, better machines, or drip irrigation—things that could make a real difference. They just keep going in circles, stuck in the same problems, and get even more at risk when bad weather, price drops, or pests come around. It's not only about losing money — it hits them emotionally too.

People don't really talk enough about how all these constant rejections and money problems mess with farmers' minds and feelings. Many feel helpless or lose hope in formal systems altogether. In extreme cases, we've even seen farmer suicides—specially in places like Maharashtra, Punjab, Telangana, and Karnataka—which makes it very clear that something has to change.

Right now, the financial system isn't working for the small farmer. Fixing this problem ain't just about giving out more loans — it needs stuff that's easy for farmers to use, better technology, and actual support they can trust. Loans should be simple to understand, quick to get, and really helpful for farmers' needs.

That's where FarmEasy comes in. It's a digital platform built especially for farmers—not for paperwork or middlemen—but to genuinely help them out. With handy digital tools, automatic eligibility checks, easy loan comparisons, info on government subsidies, and simple guides in local languages, FarmEasy wants to bring back trust in financial services. It

ain't just about applying for loans — it's about showing farmers the ropes, helping them learn, and actually supporting them right.

With FarmEasy, the whole point is simple:

👉 No more waiting forever.

👉 No more getting lost or confused.

👉 Just quick, fair loans made for farmers — right when and where they need him.

1.3. Feasibility Study

This study checks if FarmEasy can actually work and be helpful for farmers and lenders. We looked at the tech side, costs, how it will run day-to-day, legal rules, and how it will affect people.

1. Tech Stuff

FarmEasy gotta have strong tech, right? It's gotta do a lot — check if farmers can get loans real quick, keep their login safe, talk with banks and govt websites, and give updates right away. The system's built with new tools so it don't slow down even if lots of people use it at the same time.

The part farmers see is made simple so even those who don't use computers much can handle it easily. It works on phones too, which is important since many farmers use smartphones now.

To share info with banks and government, FarmEasy talks to their systems securely. All the important info is kept super safe — it's scrambled using techy security stuff so no one can just sneak in and grab it.

They used everyday, easy-to-work-with tech to build it, which means if something goes wrong or they wanna add new stuff like more banks or loan types, they can do that without breaking anything or starting from scratch.

2. Money Side

FarmEasy should save money for everyone. Farmers don't have to travel far or pay middlemen to get loans, so they save time and cash. Banks spend less on paperwork and

manual checks too.

Building FarmEasy won't be super expensive because it uses open-source tools and cloud servers that grow with demand. Keeping it running is also cheaper thanks to automation.

If farmers get loans on time, they can buy better seeds and tools, grow more crops, and earn more money. That helps cut poverty in villages and boosts local economies.

FarmEasy also makes it harder for fraud or late payments, which banks like.

3. Running It Day-to-Day

How good FarmEasy works really depends on how easy it is for farmers to use. The design is kept simple and walks them through each step.

It gets rid of lots of middlemen and piles of paperwork, making everything quicker for everyone involved. Farmers get loan updates right away, so no need to keep calling or visiting banks.

At first, farmers might need some training to get used to the site. But once they get the hang of it, they won't need much help.

The system's pretty easy to update when new loans or government schemes pop up — no big deal.

4. Legal stuff

FarmEasy keeps lots of private info, so it's gotta follow all the rules to keep things safe. It scrambles the data so nobody can steal it, and always asks farmers if it's okay to use their info.

They only work with banks that got proper licenses, and they really check to make sure the users are who they say they are — no cutting corners or cheating.

Also, they stick to all the government's rules about online loans and moving money, so banks and lenders trust the whole setup without any worries.

5. Social stuff

FarmEasy mainly helps small farmers — the ones who find it toughest to get loans. Making loans easier means less gap between rich and poor farmers and more farmers joining the official system.

Simple loan process means farmers don't get stressed out and can track loans easily, so they trust banks more and bad middlemen can't cheat them.

When farmers get to invest more, farming gets better, jobs pop up, and villages get stronger. If FarmEasy grows, it can really change lives in poor places.

1.4. Project Planning

Effective planning is the foundation of any successful project. For FarmEasy, a structured and well-thought-out approach ensures that the platform is delivered on time, meets user needs, and uses resources wisely. This section outlines the core goals, features, development strategy, and team responsibilities.

1. Objectives of the Project

FarmEasy is designed with the following aims:

- Build a unified online portal where farmers can explore and apply for different loan schemes.
- Streamline the entire loan process, making it quicker and easier to navigate.
- Provide a reliable, secure environment for farmers, banks, and government bodies to interact.
- Eliminate the need for middlemen and enhance access to timely financial assistance.

2. Scope of the Project

Features Included in This Release:

- Secure login and registration system with role-specific access (for farmers, banks, and administrators)
- A dashboard to compare and apply for available loan plans
- Online loan application form submission

- Automatic eligibility assessment based on scheme requirements
- Instant updates on the progress of each application
- API integration with banking and government platforms
- Administrative tools for monitoring and validating activity
- Data protection through built-in security measures

🚫 Features Not Included Yet:

- Multilingual support (planned for future enhancements)
- Offline support or native mobile app compatibility
- Insurance or subsidy modules tied to loan programs

3. Step-by-Step Development Plan

FarmEasy follows an Agile development model, allowing for flexibility, user feedback, and regular improvements. Here's a breakdown of how development will proceed:

a) Requirement Gathering

- Conduct discussions with end-users (farmers) and stakeholders (banks, officials) to identify pain points and expectations.
- Define all necessary functionalities and prepare detailed specifications.
- List external APIs and tools required for integration.

b) System Design

- Design system architecture covering frontend, backend, database schema, and user access levels.
- Build UI wireframes for different user roles.
- Choose the best-suited tech stack based on functionality, scalability, and team expertise.

c) Core Development

- Frontend: Build responsive user interfaces accessible on desktop and mobile browsers.
- Backend: Develop main logic for authentication, application management, and business rules.
- API Layer: Integrate with external financial or governmental systems.
- Security: Add user authentication, role-based access, and secure data handling mechanisms.

d) Quality Assurance

- Perform unit tests for all core components.
- Carry out integration and system-level testing.
- Onboard a test group of users (farmers) for real-world feedback.
- Address any issues or usability concerns before full deployment.

e) Deployment & Launch

- Host the platform using a secure, scalable cloud service.
- Maintain two environments: one for internal testing and one for live users.
- Configure automatic backups and disaster recovery tools.

f) Maintenance & Support

- Monitor application health and user activity continuously.
- Provide regular updates to enhance performance and fix bugs.
- Ensure timely security patches and improvements based on user feedback.

4. Team Structure & Technology Stack

Project Roles:

- Product Manager
- Frontend Developer
- Backend Developer

- UX/UI Designer
- QA Engineer
- Integration Developer
- Customer Support Specialist

Tools & Technologies:

- Frontend: React.js, HTML5, CSS3, Bootstrap 5
- Backend: Spring Boot (Java)
- API Format: REST (JSON-based)
- Database: MariaDb
- Source Control: Git (GitHub)

1.5. Project Planning

I. PERT Chart (Program Evaluation Review Technique)

PERT is useful for identifying the sequence of tasks, dependencies, and the critical path of the project. Below is the breakdown of tasks and their estimated durations.

Table 1 Task Breakdown with IDs:

Task ID	Task Description	Predecessor(s)	Duration (Days)
A	Requirement Analysis	—	7
B	System Design	A	9
C	Frontend Development	B	12
D	Backend Development	B	15
E	API Integration	C	5
F	Security and Authentication Setup	D, C	4
G	Testing (Unit + Integration)	F	5

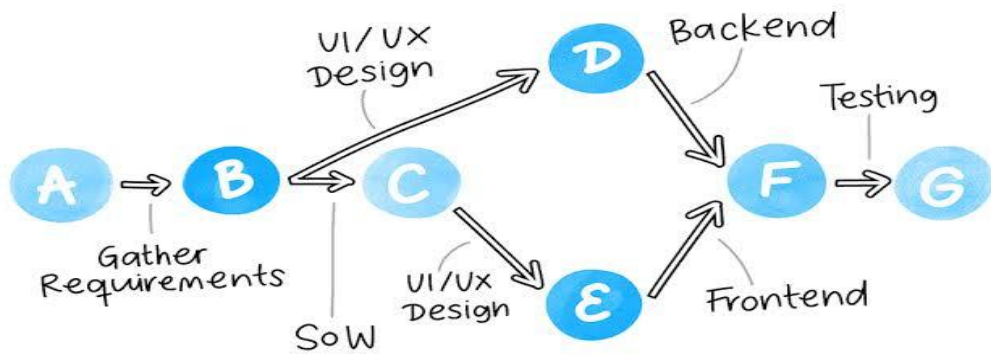


Figure 1 PERT Chart

Figure 1 represents the PERT chart for the project planning phase of the project FarmEasy.

II. Gantt Chart (Tabular Representation)

The Gantt chart represents the timeline of the project linearly. Here's how the project looks over a 9-week schedule (approx. 57 working days):

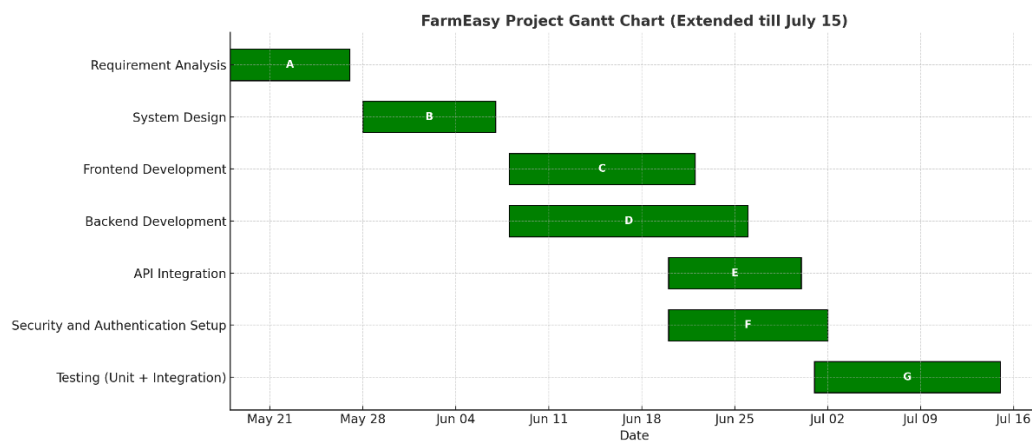


Figure 2 Gantt Chart

Figure 2 represents Gantt chart for the project “Farmeasy” according to the week and days given on each and every phase of the project.

Software Requirements Specifications (SRS)

2.1. Software Engineering Paradigm Applied In FarmEasy

- i. Agile Development Paradigm
 - a. Why: FarmEasy handle changing user requirements and a diverse range of users — farmers, banks, and administrators — while also integrating with external systems like banks and government database. Agile support iterative development, continuous feedback and quick adaptability to these changes, which is crucial for delivering value and incorporate feedback, especially from rural and diverse users.
 - b. How: Iterative sprints is used to develop core features such as registration, loan application and eligibility checking. Frequent releases helps testing usability and bring improvements. There's close collaboration with stakeholders including farmers and financial institutions.
- ii. Component-Based Development (CBD):
 - a. Why: FarmEasy consists of multiple independent modules like user authentication, loan eligibility engine, document management, subsidy information, notification system, and financial literacy contents. CBD allow these modules to be developed, tested and maintained separately which improve scalability and ease of management.
 - b. How: Separate microservices or components are built for each major functions. Clear API contracts between components ensures smooth integration.
- iii. Client-Server Paradigm:
 - a. Why: FarmEasy is a web and mobile platform where users (clients) access services hosted on backend servers which handles database, business logic, and integrations. This paradigm supports centralized management and data security.
 - b. How: Frontend UI as client apps/web interface. Backend RESTful APIs and database servers.
- iv. Service-Oriented Architecture (SOA):

- a. Why: To integrate with external banking systems, credit bureaus, and government subsidy APIs, FarmEasy adopts SOA principles to enable interoperability and reusability.
 - b. How: Expose core services (eligibility checks, application status, subsidy info) as independent, loosely coupled services. Use standard communication protocols (HTTP/HTTPS, JSON/XML).
- v. Event-Driven Architecture (EDA) (Optional/Complementary):
 - a. Why: For real-time notifications (application status updates via SMS/email/app), EDA supports asynchronous communication and responsiveness.
 - b. How: Event queues or message brokers usually trigger the notifications whenever the loan application status changes.

FarmEasy mostly follows Agile for its iterative development and uses component-based approach for modular design. It's built on a client-server model. The system also connects with external services using SOA and can make use of event-driven architecture especially for notification purposes.

2.2. Feature Prioritization and Finalization

Using a Weighted Scoring Model, features were evaluated based on their Impact, Feasibility, Cost, and Time to Implement.

Table 2 Feature Prioritization and Finalization

Feature	Priority	Reason for Selection
Application Creation	High	Core entry point for farmer participation
Loan and Scheme Browsing	Medium	Essential for facilitating financial access
Loan Eligibility Check	High	Immediate feedback tool increasing engagement
Grievance Reporting	Medium	Increases trust in the platform
Application Status Monitoring	High	Essential for transparency and user satisfaction
Launch of Loan Schemes by Banks	Medium	Required to keep the system dynamic and updated
Government Oversight and Suppression Control	Medium	Critical for governance, transparency, and scheme evaluation

2.3. Design Flow

FarmEasy 360° uses a flexible and scalable modular layered architecture, which is designed to integrate smoothly with the existing agricultural and financial systems. The development lifecycle is organized into five main phases to ensure smooth functionality and to provide the best user experience for smallholder farmers and lending institutions.

1. Data Collection Layer:

- a. It aggregates data from multiple sources like weather, soil health, farmer demographics, land records, and transaction history.
- b. The data sources includes agritech partners, government databases and financial institutions.
- c. Data can be temporarily stored locally on user devices and then synchronized with cloud when possible.

2. Data Preprocessing and Validation Layer:

- a. This layer cleans, structures and enrich the raw data to make it consistent and usable.
- b. It validates land ownership records.
- c. Also processes crop images to find patterns and any anomalies.
- d. It handles missing data, standardizes regional formats such as dates and addresses, and normalizes entries in multiple languages.

3. Intelligence and Analytics Layer: This is the core system where AI and ML models analyzes the data for different tasks:

- a. Predicting loan eligibility.
- b. Monitoring weather forecasts and crop cycles using time-series models to estimate repayment capacity.
- c. Segmenting borrowers based on creditworthiness and risk by using clustering algorithms.
- d. The system also ensures transparency by providing explanations that users and regulators can understand regarding credit decisions.

4. User Interface Layer: This layer supports several interaction modes to make the system inclusive:

- a. Web interfaces to give wider access.
- b. Mobile apps with dashboards for users comfortable with digital tools.

5. Backend Infrastructure Layer:
 - a. It manages secure user authentication, session management, along with encrypted data storage.
 - b. APIs are provided to connect in real-time with NBFCs, cooperative banks, insurance providers and other financial services.
 - c. Unique transaction IDs are assigned for all user sessions and loan transactions, which help ensure audit trails are kept properly.
6. Feedback and Learning Loop:
 - a. Once decisions been made, the system engage users by using AI-driven behavioral predictions like reminders for repayments, farming tips, and insurance updates.
 - b. It keeps on collecting feedback and fresh data continuously to retrain AI models, so accuracy and personalization improve over time.
 - c. This improves user engagement and system adaptability over time.
7. Architectural Highlights:
 - a. The architecture uses microservices where each functional module (credit scoring, farmer profiling, insurance, etc.) works independently and can be updated or scaled without affecting others.
 - b. It's designed to handle India's diverse infrastructure, languages and regulatory environments especially in rural areas.
 - c. Enables scalable, transparent and interpretable agrifinance delivery in near real-time.

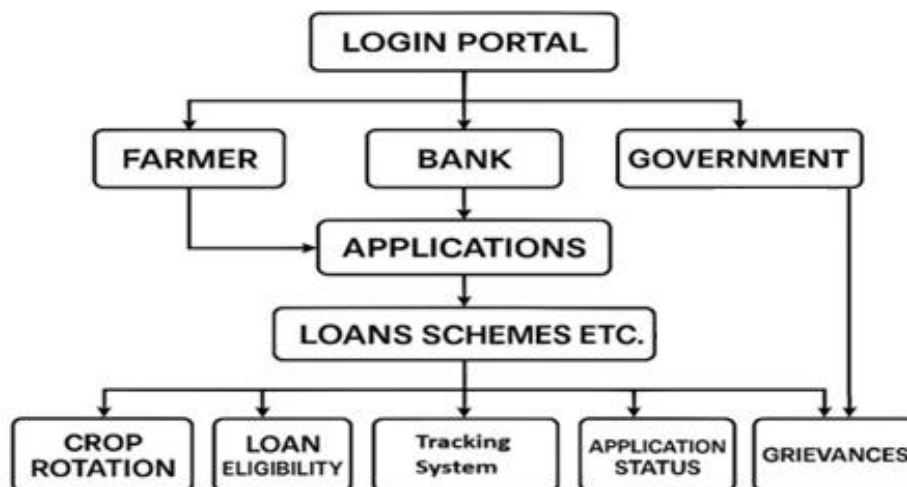


Figure 3 Data Flow Diagram (DFD)

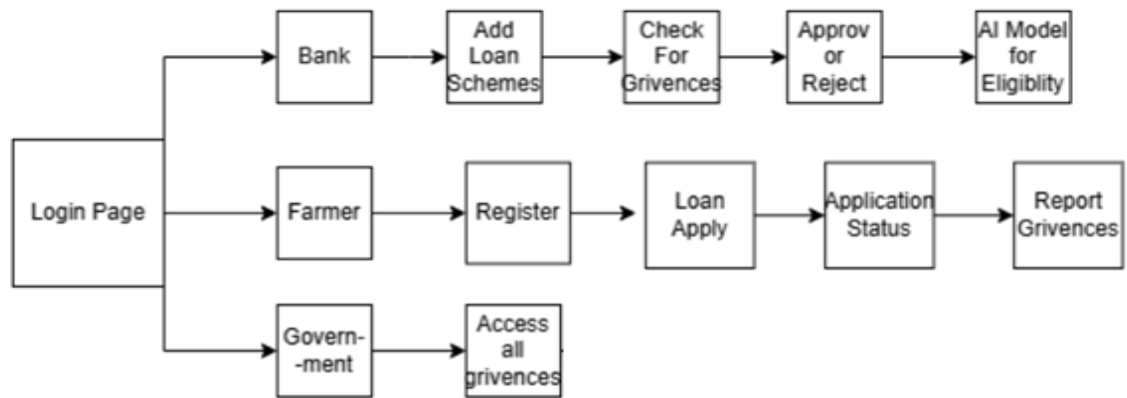


Figure 4 Main Flow Diagram

System Design

3.1. Modularisation Details:

3.1.1. User Management Module

Purpose: Handle authentication, authorization, and user roles.

- Features:
 - Secure login/register (JWT-based)
 - Role-based access (Farmer, Admin, Bank, Govt)
 - Profile update & password reset
- Entities:
 - User
 - Role
- Tech: Spring Security, JWT, BCrypt

3.1.2. Farmer Support Module

Purpose: Provide user-friendly access for farmers.

- Features:
 - Help content for non-tech users
 - Farmer dashboard (loans applied, status, messages)
 - Localization (support for Hindi/local languages)
- Entities:
 - FarmerDetails
 - SupportQuery

3.1.3. Loan Management Module

Purpose: Centralized loan discovery, application, and status tracking.

- Features:
 - View and compare loans (Govt, Bank, NBFC)
 - Apply for loan (single form across all sources)

- Track application status
- Entities:
 - LoanScheme
 - LoanApplication
- Logic: Integrates with rule engine (.drl or internal) to check eligibility

3.1.4. Bank & Government API Integration Module

Purpose: Connect directly with financial institutions.

- Features:
 - API connection with partner banks/government DBs
 - Auto fetch credit score, bank details, application updates
- Entities:
 - BankPartner
 - BankResponse
 - LoanStatusUpdate

3.1.5. Frontend Module

Purpose: Simple, intuitive UI for all users.

- Farmer Side:
 - Guided form filling
 - Language toggle
 - Progress bar for application
- Admin/Bank Side:
 - Loan application queue
 - Verification checklist
- Tools: React + TailwindCSS for server-side rendering

Modular Communication

- **Internal communication:** REST APIs between backend modules
- **External communication:** REST with banks/government

3.2 Data Integrity and Constraints

The constraints determine how and whether FarmEasy could be set up. In the usual way, these will be technical, legal, financial, and end-user constraints that might affect architecture, scalability, functionality, or deployment. Therefore, it is essential in real-world planning and sustainable implementation to know and appreciate these constraints.

(a) Technological Constraints

(i) Internet & Device Accessibility:

- Many of the target users are settled in places where one can talk about unreliable internet connectivity, typically, remote or semi-urban places.
- Users may constitute a considerable percentage who own low-end smartphones or basic feature phones.
- Design Response: Prioritize lightweight mobile applications, offline data entry modes, SMS-based services, and progressive web apps.

(ii) Data Availability & Quality:

- Regions elsewhere often do not have correct, labeled, and fine-grained agricultural and financial data.
- Heterogeneity among datasets (back and forth with, say, crop types, regional dialects, and loan terms), invariably causes complications in model training.
- Design Response: Use self-supervised and transfer learning techniques for data independence.

(b) Financial Restrictions

(i) Cost of Development and Deployment:

- With a tight budget, it can become difficult or even not possible to afford commercial-grade APIs, expensive tools, or paid cloud platforms for hosting or scaling.
- Design Response: Better to rely on open-source libraries, try to stick with free-tier APIs, and use modular architecture to manage costs more flexibly.

(ii) Affordability of End-Users:

- Most small-scale farmers may not afford things like app subscriptions, data usage costs or even fees attached to loan processing.
- Design Response: The core system should be totally free for users. Costs can be recovered later through partnerships — like from NBFCs, agri-input firms or banks.

(c) Regulatory & Compliance Boundaries

(i) KYC & Data Privacy Regulation:

- The platform needs to comply with data laws in India — like DPDP Bill, RBI data norms, and if needed, some international privacy standards too.
- Also, Aadhaar, PAN numbers, biometrics, and credit scores must be stored and processed in a secure way.
- Design Response: Strong encryption like AES-256, anonymizing datasets used in AI models, and permission-based data sharing should be used to meet privacy and security goals.

(ii) Banking & Financial Sector Regulations:

- Digital lending must follow rules given by the RBI, along with any specific lending norms applicable to certain areas.
- Design Response: Maintain audit logs, offer dashboards for checking compliance, and include built-in checks to prevent actions that go against policy or law.

(d) Environmental & Infrastructural Constraints

(i) Power & Internet Access Issues:

- In several rural areas, electricity isn't always reliable, and network connections can be patchy or extremely slow. This poses a problem for platforms that rely on constant internet access to sync user data or provide real-time services.
- Human-Focused Solution: To tackle this, the app should work even when offline by saving data locally and syncing it automatically once a stable connection is available. It should also run efficiently on basic smartphones without draining too much battery — specially since power cuts are quite common in many areas.

(ii) Unpredictable Weather & Crop Risk:

- Farming outcomes are directly linked to weather, which is often very inconsistent. Unseasonal rains or long dry spells can ruin the harvests, making it difficult for farmers to repay their loans.
- Human-Focused Solution: Rather than relying on outdated trends, the system needs to adjust regularly. Models should keep learning from new environmental data — including weather updates, pest outbreaks, and soil changes — so that credit risk evaluations remain grounded in reality.

(e) Challenges Faced by Real People

(i) Limited Digital Skills & Language Barriers:

- Human-Focused Solution: The platform should feel easy to use and welcoming, not something that overwhelms or scares them. So, it should rely more on pictures or simple icons, and offer voice instructions in several regional languages. Also, it should guide users step-by-step, so even those who never used an app before can navigate without feeling lost or frustrated.
- Many farmers still don't have much knowledge about technology. Even those who own smartphones, often find apps confusing — especially when instructions are written in English or very formal Hindi, which isn't how they usually speak in their local language or dialect.

3.3. Database Design

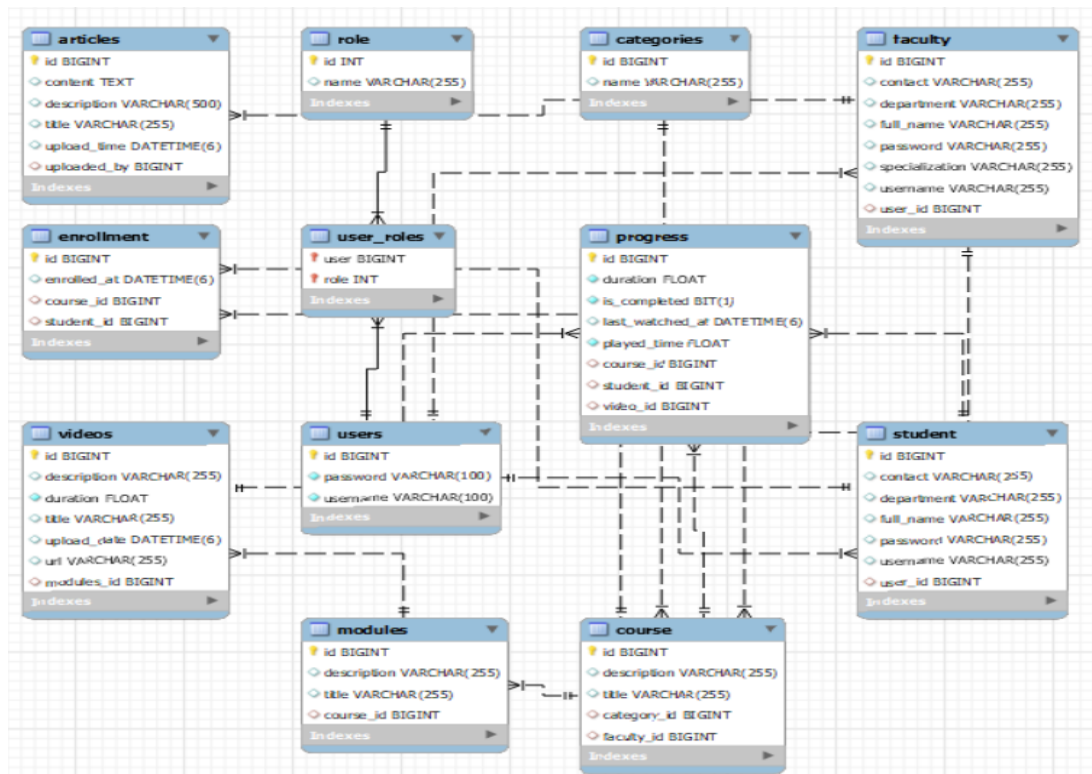


Figure 5 Schema or Database Desgin

3.4. User Interface Design

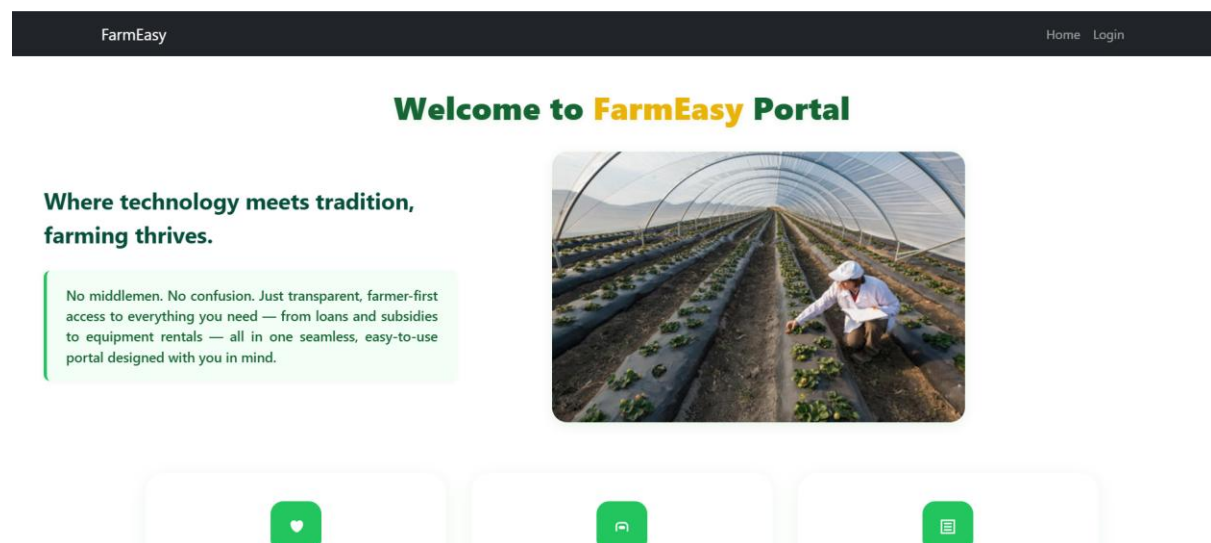


Figure 6 Home Page of the Portal

Farmer Registration

Registration successful! Redirecting to login...

Name

Priyanshu Baral

Contact

9876543210

Address

H.No.-62, Street No.-6 Block C

Email

priyanshu@gmail.com

Password

.....

Register

Figure 7 Farmer Registration

Login

Login successful! Redirecting... X

Username

priyanshu@gmail.com

Password

.....

Login

Don't have an account? [Register here](#)

Figure 8 Farmer Login

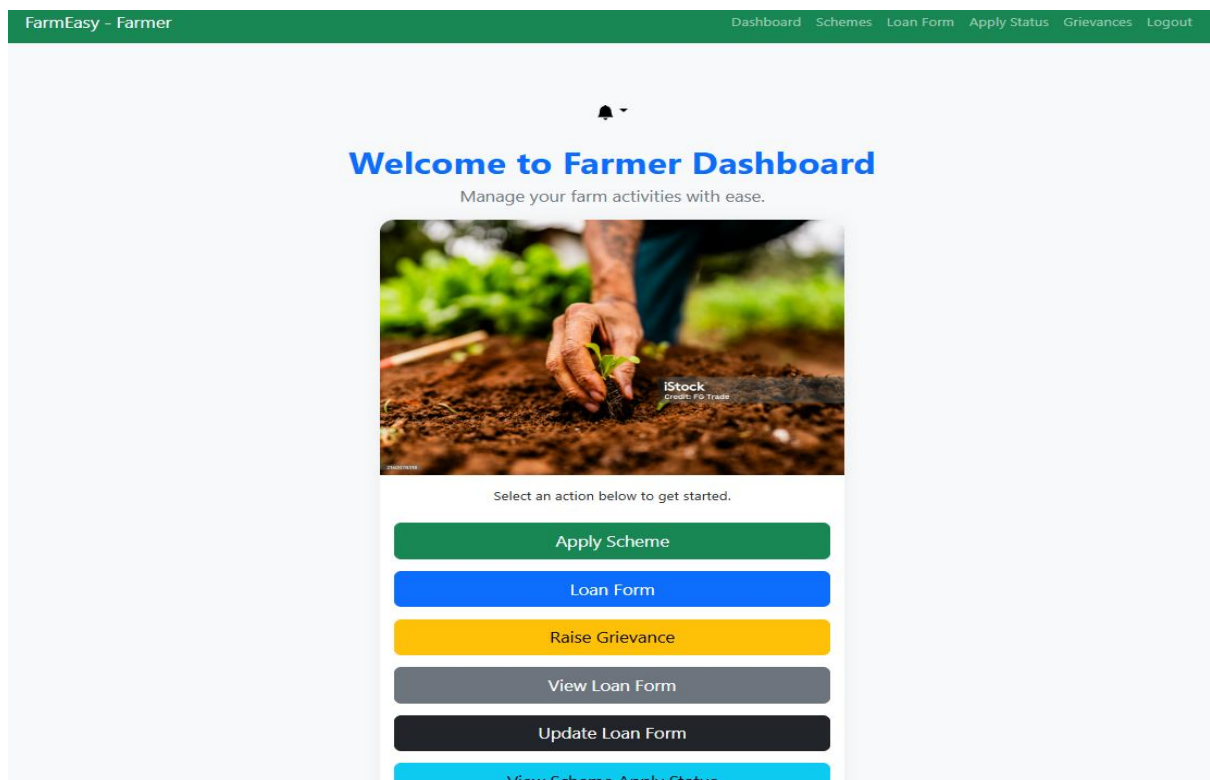


Figure 9 Farmer DashBoard

The screenshot shows the 'Farmer Loan Application Form' in the FarmEasy system. The header is green with the same navigation links as Figure 9. The form title is 'Farmer Loan Application Form' with a coin icon. The form fields are arranged in two columns. Left column: Name (Priyanshu Baral), Contact (9876543210), CIBIL Score (750), Address (H.No.-62, Street No.-6 Block C), District (North East), Aadhaar Number (784521453256). Right column: Email (priyanshu@gmail.com), Gender (Male), Age (23), Salary (₹) (30000), State (New Delhi), PIN Code (110094), and Aadhaar PDF Upload (Choose File, Untitled.pdf).

Figure 10 Farmer Application Form

FarmEasy - Bank

DashboardMy SchemesAdd SchemeScheme RulesLoan ApplicationsGrievancesLogout

Add New Scheme

Scheme Name

Krishi Samarth Loan

Scheme Code

1001

Description

A government-backed scheme offering financial support to small and marginal farmers for purchasing equipment and seeds.

Benefits

Low interest rates, flexible repayment options, and subsidy for early r

Eligibility

Farmer must own less than 5 acres of land and have a minimum CIBIL

Documents

Min Salary

20000

Max Salary

300000

CIBIL Score

700

Rate of Interest

6.5

Tenure

5

Scheme Type

Agriculture

Figure 11 Bank adding new Scheme

FarmEasy - Bank

DashboardMy SchemesAdd SchemeScheme RulesLoan ApplicationsGrievancesLogout

Bank Dashboard

Total Schemes

1

Loan Applications

0

Approved Loans

0

Pending Requests

0

+ Add New Scheme

View Loan Applications

Manage Scheme Rules

View Grievances

Your Created Schemes

Krishi Samarth Loan

Agriculture

ROI: 6.5%

Tenure: 5 months

Description: A government-backed scheme offering financial support to small and marginal farmers for purchasing e...

EditDelete

Figure 12 Bank Dashboard

 **Create New Scheme Rule**

Rule Name

Approve if cibil score is greater than 700

Scheme ID

1001

Field

cibilScore

Operator

>

Value

700

Create Rule

Figure 13 Creating Rules for the schemes

FarmEasy - Farmer

Dashboard Schemes Loan Form Apply Status Grievances Logout

Available Loan Schemes

You have applied for the Krishi Samarth Loan from State Bank of India.

All BanksAll Scheme Types

Krishi Samarth Loan

State Bank of India

Tenure: 5

ROI: 6.5

Description: A government-backed scheme offering financial support to small and marginal farmers for purchasing e...

100000

Apply

View Details

Figure 14 Loan schemes available for the farmers

Loan Applications

#	Farmer Name	Loan Form	Scheme	Amount	Status	Review	Actions
1	Priyanshu Baral	View Form	Krishi Samarth Loan	₹100000	PENDING	Approved	Approve Reject

Figure 15 Application form visible to the bank

My Application Status

#	Scheme	Bank	Amount	Applied On	Status	Reviewed On	Review Comment
1	Krishi Samarth Loan	State Bank of India	₹100000	20 Jul 2025	APPROVED	20 Jul 2025	Approved

Figure 16 Status of the application form for the farmer

Submit Grievance

Grievance Type

Loan Delay

Description

The Bank delayed in loan approval

Select Bank

State Bank of India

Submit Grievance

Figure 17 Report for the Grievances

FarmEasy - Government
Dashboard Grievances Logout

Government Grievance Dashboard

Farmer	Bank	Date	Type	Description	Status	Review	Action
Priyanshu Baral	State Bank of India	2025-07-20	Loan Delay	The bank delayed in load approval	RESOLVED	Reviewed	Save

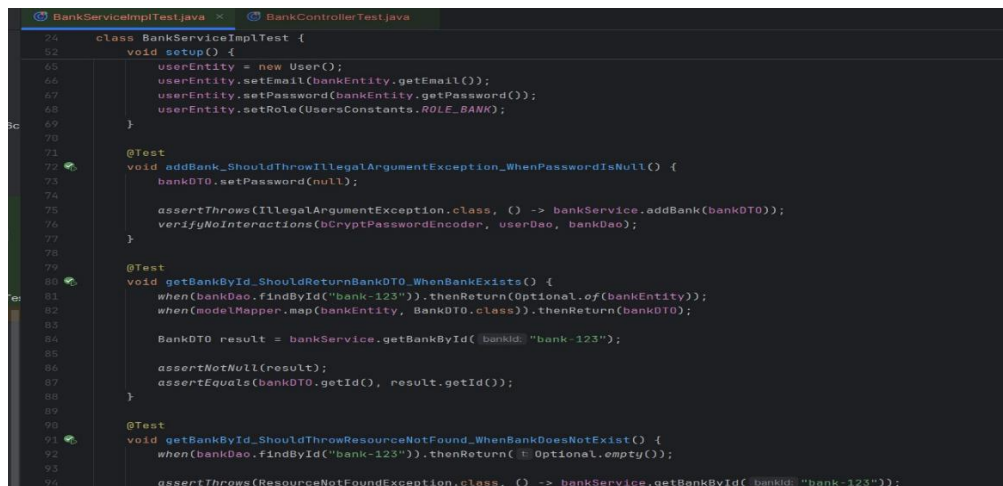
Figure 18 Process on the reported grievances

3.5. Test Cases (Unit Test cases and System Test cases)

- Unit Test Cases – test individual components (like services, controllers).
- System Test Cases – test the whole system workflow (end-to-end use cases).

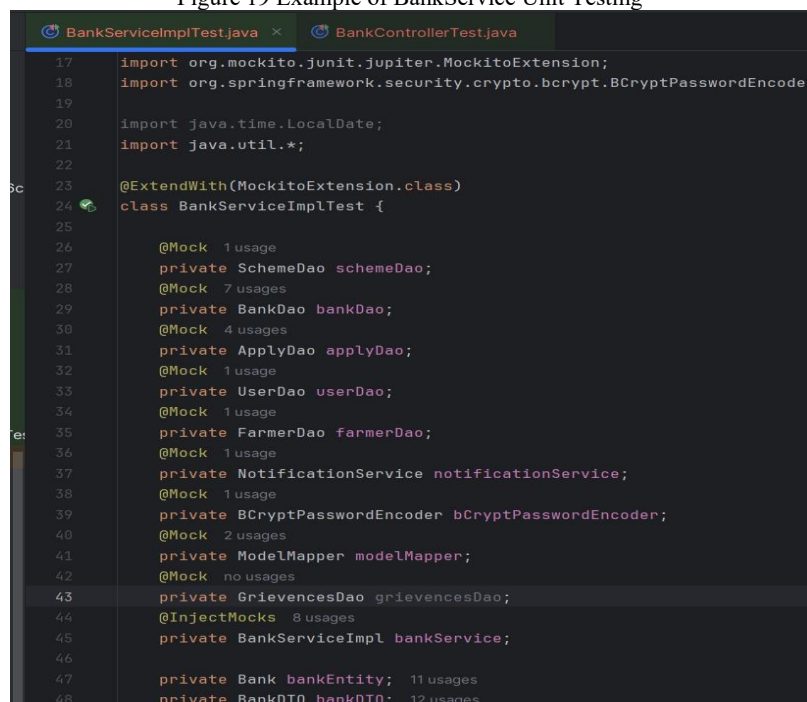
3.5.1. UNIT TEST CASES (Using JUnit + Mockito – Spring Boot)

1. BankServiceImpl



```
24 class BankServiceImplTest {
52     void setup() {
65         userEntity = new User();
66         userEntity.setEmail(bankEntity.getEmail());
67         userEntity.setPassword(bankEntity.getPassword());
68         userEntity.setRole(UsersConstants.ROLE_BANK);
69     }
70
71     @Test
72     void addBank_ShouldThrowIllegalArgumentException_WhenPasswordIsNull() {
73         bankDTO.setPassword(null);
74
75         assertThrows(IllegalArgumentException.class, () -> bankService.addBank(bankDTO));
76         verifyNoInteractions(bCryptPasswordEncoder, userDao, bankDao);
77     }
78
79     @Test
80     void getBankById_ShouldReturnBankDTO_WhenBankExists() {
81         when(bankDao.findById("bank-123")).thenReturn(Optional.of(bankEntity));
82         when(modelMapper.map(bankEntity, BankDTO.class)).thenReturn(bankDTO);
83
84         BankDTO result = bankService.getBankById(bankId: "bank-123");
85
86         assertNotNull(result);
87         assertEquals(bankDTO.getId(), result.getId());
88     }
89
90     @Test
91     void getBankById_ShouldThrowResourceNotFoundException_WhenBankDoesNotExist() {
92         when(bankDao.findById("bank-123")).thenReturn(Optional.empty());
93
94         assertThrows(ResourceNotFoundException.class, () -> bankService.getBankById(bankId: "bank-123"));
```

Figure 19 Example of BankService Unit Testing



```
17 import org.mockito.junit.jupiter.MockitoExtension;
18 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
19
20 import java.time.LocalDate;
21 import java.util.*;
22
23 @ExtendWith(MockitoExtension.class)
24 class BankServiceImplTest {
25
26     @Mock 1 usage
27     private SchemeDao schemeDao;
28     @Mock 7 usages
29     private BankDao bankDao;
30     @Mock 4 usages
31     private ApplyDao applyDao;
32     @Mock 1 usage
33     private UserDao userDao;
34     @Mock 1 usage
35     private FarmerDao farmerDao;
36     @Mock 1 usage
37     private NotificationService notificationService;
38     @Mock 1 usage
39     private BCryptPasswordEncoder bCryptPasswordEncoder;
40     @Mock 2 usages
41     private ModelMapper modelMapper;
42     @Mock no usages
43     private GrievencesDao grievencesDao;
44     @InjectMocks 8 usages
45     private BankServiceImpl bankService;
46
47     private Bank bankEntity; 11 usages
48     private BankDTO bankDTO; 12 usages
```

Figure 20 Unit testing for BankServiceImpl with the help of Mokito

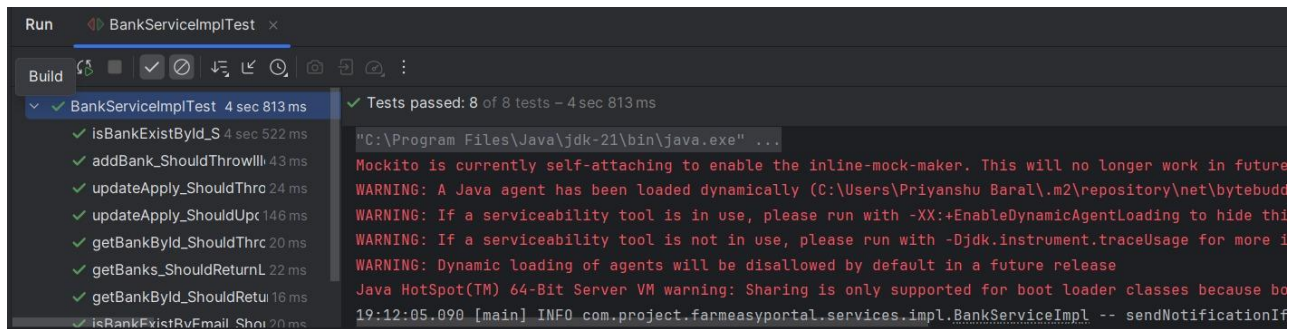


Figure 21 Unit Test cases passed

3.5.2 SYSTEM TEST CASES

1. BankControllerTest

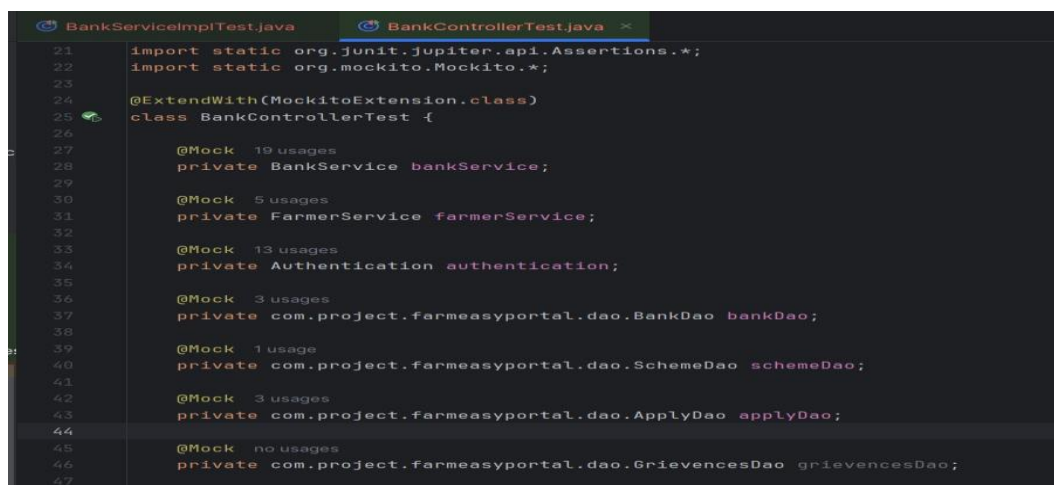


Figure 22 BankController System Testing with Mokito

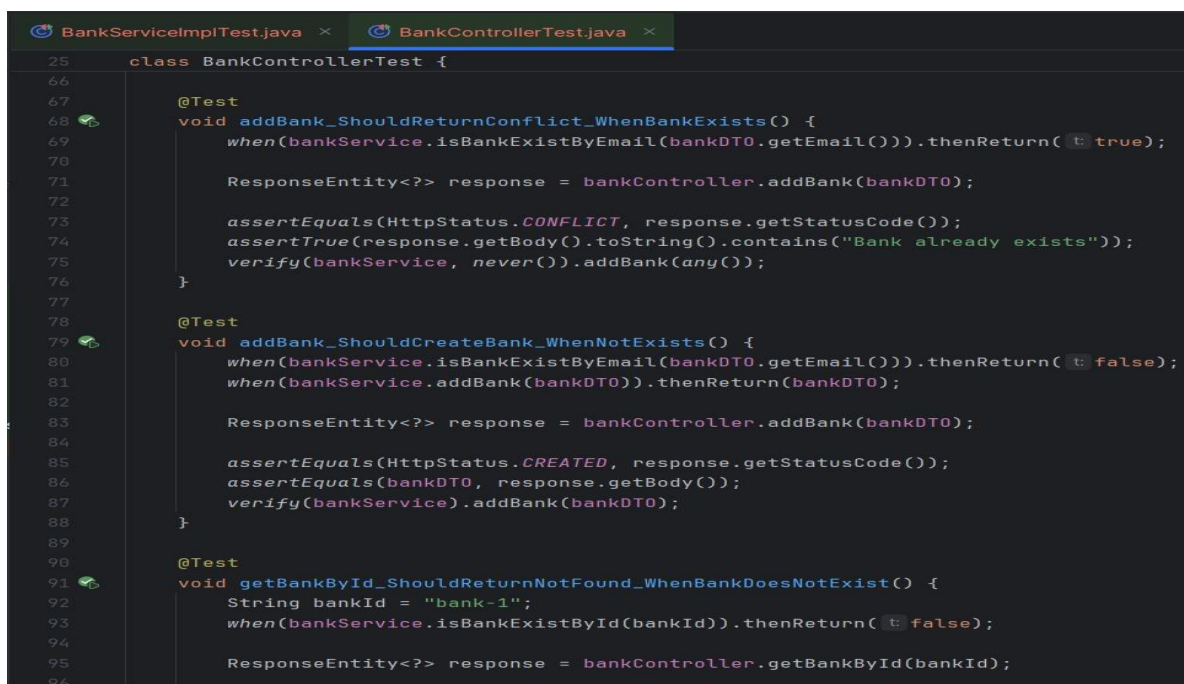


Figure 23 BankControllerTest Code

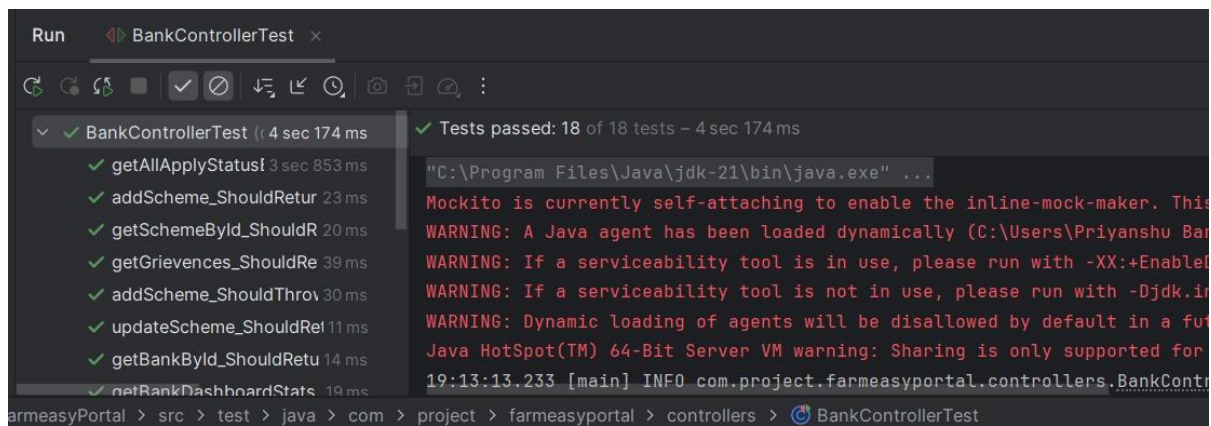


Figure 24 Test cases passed for the System test cases

Coding

4.1 . SQL Commands

```
1  ✓ spring:
2  ✓   application:
3     name: FarmeasyPortal
4
5  📖 ✓   datasource:
6     url: jdbc:mariadb://localhost:3308/farmeasy-app?createDatabaseIfNotExist=true
7     username: root
8     password: mariadb12345
9     driver-class-name: org.mariadb.jdbc.Driver
10    ✓   # Uncomment the following line if needed
11        # jpa:
12        #   hibernate:
13        💡 #     ddl-auto: update
14
```

Figure 25 Creation and connection of Database

```
1  ✓ CREATE TABLE user (
2     id          int NOT NULL AUTO_INCREMENT,
3     email       varchar(50) DEFAULT NULL,
4     password    varchar(255) DEFAULT NULL,
5     role        varchar(30) DEFAULT NULL,
6     PRIMARY KEY (id)
7 ) ENGINE=InnoDB;
```

Figure 26 Creating User table

```
1  ✓ CREATE TABLE bank (
2     id          varchar(255) PRIMARY KEY,
3     bank_name    VARCHAR(255),
4     bank_address VARCHAR(255),
5     bank_city    VARCHAR(255),
6     bank_state   VARCHAR(255),
7     bank_zip     VARCHAR(255),
8     email        VARCHAR(255),
9     bank_phone   VARCHAR(255),
10    password     VARCHAR(255)
11 ) ENGINE=InnoDB;
12
```

Figure 27 Creating the Bank table


```

1  INSERT INTO role (id, roleType)
2  VALUES (1, 'ROLE_GOV'),
3          (2, 'ROLE_BANK'),
4          (3, 'ROLE_FARMER'),
5          (4, 'ROLE_MERCHANT');

```

Figure 28 Insertion of Role

```

106 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
107     http.csrf(AbstractHttpConfigurer::disable);
108
109     http.authorizeHttpRequests(request -> {
110         request.requestMatchers(@"/api/v1/gov/**").hasRole("GOV");
111         request.requestMatchers(HttpMethod.GET).permitAll();
112         request.requestMatchers(@"/api/v1/bank/**", @"/api/rules/create",
113             @"/api/v1/farmer/getFormDetail", @"/api/v1/farmer/download").hasRole("BANK");
114         request.requestMatchers(@"/api/v1/farmer/**", @"/api/v1/payments/**").hasRole("FARMER");
115         request.requestMatchers(@"/api/v1/merchant").hasRole("MERCHANT");
116         request.requestMatchers(@"/**").permitAll();
117         request.requestMatchers(@"/v3/api-docs").permitAll();
118         request.requestMatchers(PUBLIC_URLS).permitAll();
119         request.anyRequest().authenticated();
120     })
121     .formLogin(form -> form.loginPage("/login")
122         .loginProcessingUrl("/login")
123         .successHandler(new CustomSuccessHandler())
124         .failureUrl(authenticationFailureUrl: "/login?error=true"))
125     .logout(httpSecurityLogoutConfigurer -> httpSecurityLogoutConfigurer.logoutUrl("/logout").permitAll())
126     .exceptionHandling(exception -> exception.authenticationEntryPoint(jwtAuthenticationEntryPoint))
127     .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
128     .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
129
130     http.authenticationProvider(daoAuthenticationProvider());
131
132     return http.build();
133 }
134

```

Figure 29 Access Rights for the Different User like Farmers, bank and Government

4.2. Standardization of the coding

4.2.1 Project Structure Standardization:

The project follows a layered architecture that separates concerns effectively. Here's how the packages are organized:

Table 3 Project Structure Standardization

Package	Purpose
entities	Contains all JPA entity classes mapped to DB tables.
dao	Contains repository interfaces extending JpaRepository.
services	Business logic layer interfaces.
services.impl	Concrete implementations of service interfaces.
payloads	DTOs (Data Transfer Objects) for request and response handling.
exceptions	Custom exception classes and global exception handlers.
enums	Enumerations for controlled values like roles, statuses, etc.

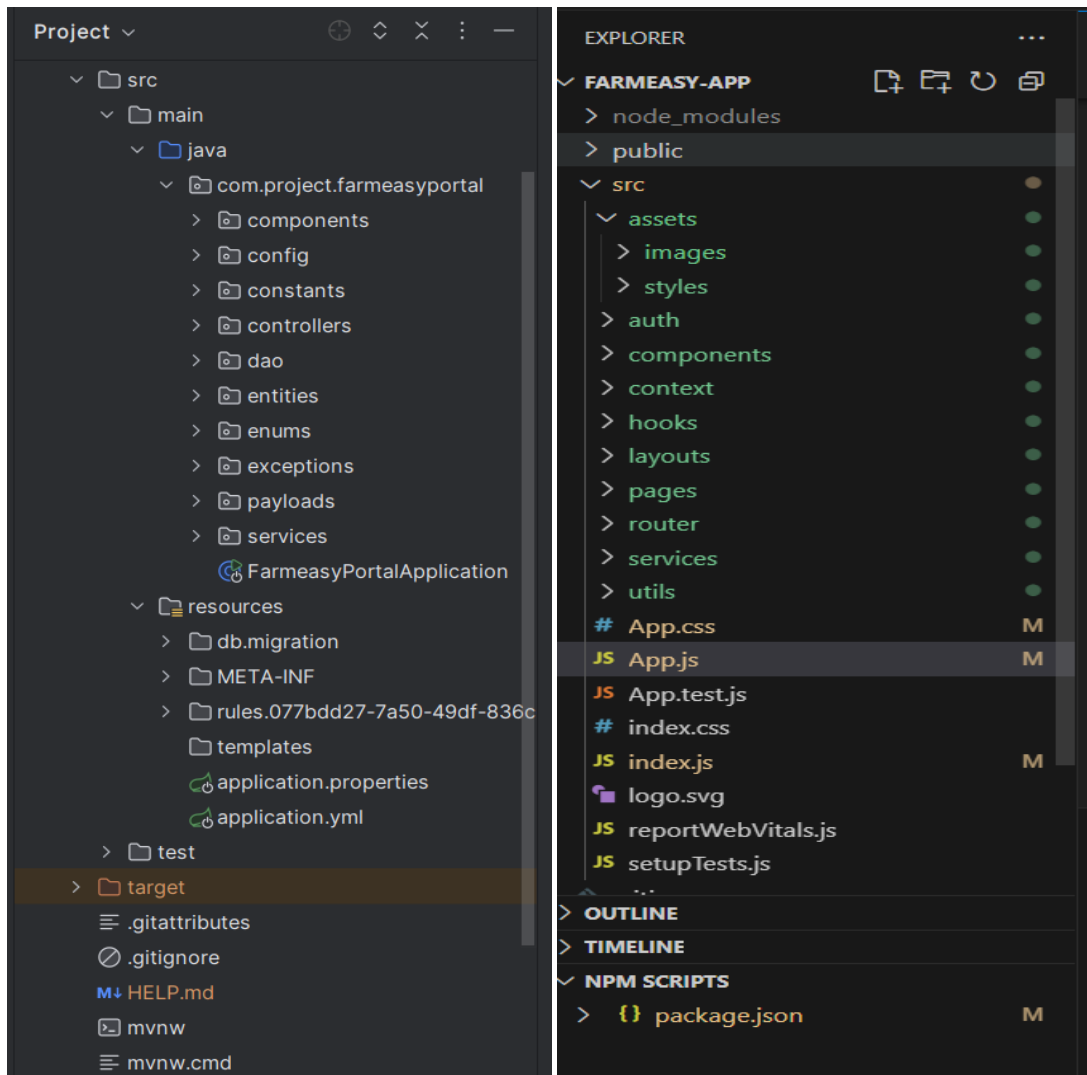


Figure 30 File Structure for the backend and frontend

4.2.2 Coding Naming Conventions:

Table 4 Example of coding naming Conventions

Component	Naming Rule	Example
Class Names	PascalCase	LoanForm, UserService
Method Names	camelCase	applyLoan(), getItemById()
Variable Names	camelCase	loanAmount, farmerId
Constants	UPPER_SNAKE_CASE	DEFAULT_ROLE
Repository Names	Suffix with Repository	LoanFormRepository
DTO Names	Suffix with DTO	LoanFormDTO, GrievanceDTO
Enum Names	PascalCase (with UPPERCASE constants)	Status.PENDING

```

package com.project.farmeasyportal.constants;

public class UsersConstants { 145 usages  ⚡ Priyanshu

    public static String USER = "User"; 4 usages
    public static String FARMER = "Farmer"; 11 usages
    public static String BANK = "Bank"; 13 usages
    public static String GOVERNMENT = "Government"; no usages
    public static String MERCHANT = "Merchant"; 15 usages
    public static String SCHEME = "Scheme"; 10 usages
    public static String APPLY = "Apply"; 2 usages
    public static String GRIEVENCE = "Grievence"; 1 usage
    public static String LOAN_FORM = "Loan Form"; 2 usages
    public static String ITEM = "Item"; 5 usages
    public static String ITEM_BOOKING = "ItemBooking"; no usages

    public static String ROLE_FARMER = "ROLE_FARMER"; 1 usage
    public static String ROLE_BANK = "ROLE_BANK"; 1 usage
    public static String ROLE_GOV = "ROLE_GOV"; no usages
    public static String ROLE_MERCHANT = "ROLE_MERCHANT"; 1 usage

    public static String DASH = "-"; 7 usages
    public static String PENDING = "Pending";

    public static String EMAIL = "email"; 11 usages
    public static String PASSWORD = "password"; no usages
    public static String ID = "id";

}

```

Figure 31 Coding Convention followed for the constraints

```

9  BankDTO addBank(BankDTO bankDTO); 2 usages 1 implementation  ⚡ Priyanshu
10 BankDTO getBankById(String bankId); 1 usage 1 implementation  ⚡ Priyanshu
11 BankDTO getBankByEmail(String email); 2 usages 1 implementation  ⚡ Priyanshu
12 List<BankDTO> getBanks(); 1 usage 1 implementation  ⚡ Priyanshu
13 Boolean isBankExistById(String bankId); 1 usage 1 implementation  ⚡ Priyanshu
14 Boolean isBankExistByEmail(String email); 2 usages 1 implementation  ⚡ Priyanshu
15

```

Figure 32 Coding Convention followed for the Methods

4.2.3 Entity Layer Standardization

- All entities reside in the entities package.
- Each entity uses proper JPA annotations like `@Entity`, `@Id`, `@Column`, etc.
- Only IDs (foreign keys) are stored in entities instead of complex nested objects (as per your preference).
- Timestamps and statuses are properly typed and handled (LocalDate, Enum, etc.)

```

1 package com.project.farmeasyportal.entities;
2
3 > import ...
12
13 @Entity 12 usages Priyanshu
14 @Data
15 @NoArgsConstructor
16 @AllArgsConstructor
17 public class Notification {
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.IDENTITY)
21     private int id;
22     private String title;
23     private String shortMessage;
24     private String fullMessage;
25     private boolean isRead = false;
26     private LocalDateTime timestamp;
27     private String farmer;
28
29 }

```

Figure 33 Standardization for the Entities

4.2.4 DTO Layer (payloads) Guidelines

- Validation annotations like @NotNull, @Size, @Pattern, etc. are used in DTOs.
- Fields in DTOs are cleanly separated from the entity to prevent overexposing internal logic.
- @JsonProperty(access = READ_ONLY/WRITE_ONLY) is used to restrict field visibility in serialization.

```

1 package com.project.farmeasyportal.payloads;
2
3 > import ...
12
13 @Getter 2 usages Priyanshu
14 @Setter
15 @AllArgsConstructor
16 @NoArgsConstructor
17 public class UserDTO {
18
19     @JsonProperty(access = JsonProperty.Access.READ_ONLY)
20     private int id;
21
22     @Email(regexp = "^[a-zA-Z0-9+_.-]+@[a-zA-Z0-9.-]+$")
23     @NotBlank(message = "E-Mail can not be empty !!")
24     private String email;
25
26     @NotBlank(message = "Password is mandatory")
27     @Size(min = 4, max = 15, message = "Password must contain minimum 5 characters and maximum of 15 characters ")
28     private String password;
29
30     private String role;
31
32 }
33

```

Figure 34 Standardization followed for the DTO

4.2.5. Service Layer Design

- Interfaces (e.g., FarmerService, GovService) are declared in the services package.
- Implementations are in services.impl for clear separation.
- Follows SOLID principles and separation of concerns.

```
1 package com.project.farmeasyportal.services;
2
3 import com.project.farmeasyportal.payloads.*;
4
5 import java.util.List;
6
7 public interface BankService { 9 usages 1 implementation  Priyanshu
8
9     BankDTO addBank(BankDTO bankDTO); 2 usages 1 implementation  Priyanshu
10    BankDTO getBankById(String bankId); 1 usage 1 implementation  Priyanshu
11    BankDTO getBankByEmail(String email); 2 usages 1 implementation  Priyanshu
12    List<BankDTO> getBanks(); 1 usage 1 implementation  Priyanshu
13    Boolean isBankExistById(String bankId); 1 usage 1 implementation  Priyanshu
14    Boolean isBankExistByEmail(String email); 2 usages 1 implementation  Priyanshu
15
16    SchemeDTO addScheme(SchemeDTO schemeDTO, String bankId); 1 usage 1 implementation  Priyanshu
17    SchemeDTO updateScheme(SchemeDTO schemeDTO, Integer schemeId); 1 usage 1 implementation  Priyanshu
18    List<SchemeDTO> getSchemesByBank(String username); 1 usage 1 implementation  Priyanshu
19    List<SchemeDTO> getSchemes(); 1 usage 1 implementation  Priyanshu
20    SchemeDTO getSchemeById(Integer schemeId); 1 usage 1 implementation  Priyanshu
21    void deleteScheme(Integer schemeId); 1 usage 1 implementation  Priyanshu
22
23    List<ApplyDTO> getApplies(); no usages 1 implementation  Priyanshu
24    List<ApplyDTO> getApplyByBank(String bankId); 1 usage 1 implementation  Priyanshu
25    void updateApply(Integer applyId, ApplyUpdateDTO applyUpdateDTO); 1 usage 1 implementation  Priyanshu
26    ApplyDTO getApply(Integer id); no usages 1 implementation  Priyanshu
27
28    List<GrievencesDTO> getGrievences(String bankId); 1 usage 1 implementation  Priyanshu
29
30 }
```

Figure 35 Service Layer Standardization

4.3. Code Efficiency

FarmEasy’s code needs to run efficiently — it should work fast, use minimal system resources, and be ready to handle a growing number of users or data without slowing down. Since the app includes different parts like managing complaints, processing loans, booking merchant products, and checking schemes, the code must not only be reliable but also stay responsive and smooth even when the workload increases.

Table 5 Approaches used for the code Efficiency

Area	Efficiency Approach
Database Operations	- Used JPA with optimized queries (JPQL or native) - Avoided N+1 problem using @Query, fetch joins, and projections
DTO Usage	Only required fields are exposed in DTOs to prevent unnecessary data

	transfer.
Validation	Used javax.validation annotations to catch errors early at the request level.
Service Logic	Heavy business logic is modularized and reused using helper methods.
Collections	Used Set instead of List where duplicates are not needed. Iterated efficiently using streams only when beneficial.
Conditional Logic	Reduced nested if-else chains by using switch-case, polymorphism, or enums where appropriate.
Code Reusability	Common methods abstracted to utility/helper classes to reduce duplication.
Exception Handling	Global exception handler avoids repetitive try-catch blocks in every controller.
Caching (Future Scope)	Identified methods like getAllSchemes() or getAvailableItems() for potential caching using Spring Cache (for performance optimization).

```

9      import jakarta.validation.constraints.Size;
10     import lombok.AllArgsConstructor;
11     import lombok.Getter;
12     import lombok.NoArgsConstructor;
13     import lombok.Setter;
14
15     @Getter 21 usages  Priyanshu
16     @Setter
17     @AllArgsConstructor
18     @NoArgsConstructor
19     public class ApplyDTO {
20
21         private int id;
22         private String farmerId;
23         private int schemeId;
24         private String bankId;
25         private LocalDate date;
26         private String statusDate;
27         private String amount;
28         private String status;
29         private String review;
30
31         // Embedded related objects
32         private FarmerDTO farmerDTO;
33         private SchemeDTO schemeDTO;
34         private BankDTO bankDTO;
35     }
36
37

```

Figure 36 Example of code efficiency

4.4. Error Handling

FarmEasy aims to catch problems early and make sure both users and developers clearly understand what's going wrong. The main objective is to smoothly fix these issues so the app keeps working without crashing. Because the platform deals with sensitive areas like loans, complaints, and merchant transactions, having trustworthy and steady error management is key to keeping the system secure and maintaining user trust.

Table 6 Mechanism for the error handling

Layer	Error Handling Mechanism
Controller Layer	Wrapped with try-catch or uses global handler to avoid code duplication.
Service Layer	Throws custom exceptions (e.g., <code>LoanNotFoundException</code> , <code>InvalidAadhaarException</code>).
Global	<code>@ControllerAdvice</code> with <code>@ExceptionHandler</code> to catch and format all errors.
Validation Layer	Uses <code>@Valid</code> + <code>@NotNull</code> , <code>@Pattern</code> , etc., with automatic binding error handling.
Security Layer	Unauthorized access returns 401, forbidden returns 403 with appropriate JSON error.

```

1 package com.project.farmeasyportal.exceptions;
2
3 > import ...
4
5
6
7
8
9
10 @ControllerAdvice
11 public class GlobalExceptionHandler {
12
13     @ExceptionHandler(ResourceNotFoundException.class)
14     public ResponseEntity<ApiResponse> resourceNotFoundExceptionHandler(ResourceNotFoundException ex) {
15         ApiResponse apiResponse = ApiResponse.builder()
16             .message(ex.getMessage())
17             .build();
18         return new ResponseEntity<>(apiResponse, HttpStatus.NOT_FOUND);
19     }
20

```

Figure 37 GlobalException Handler example code

```

1 package com.project.farmeasyportal.exceptions;
2
3 > import ...
4
5
6 @Getter
7 @Setter
8 public class ResourceNotFoundException extends RuntimeException {
9     private final String resourceName;
10    private final String fieldName;
11    private final String field;
12
13    public ResourceNotFoundException(String resourceName, String fieldName, String field){
14        super(String.format("%s not found with %s : %s", resourceName, fieldName, field));
15        this.resourceName = resourceName;
16        this.fieldName = fieldName;
17        this.field = field;
18    }
19 }
20

```

Figure 38 ResourceNotFound Custom Exception Handler

4.5. Parameter Passing/ Calling

FarmEasy depends on passing parameters to make sure different parts of the system — like controllers, services, repositories, and frontend APIs — can communicate properly. By handling these parameters carefully and securely, the system maintains smooth data flow, stays modular, and avoids issues caused by faulty or unsafe data injection.

Table 7 Techniques for the Parameter Passing/ calling

Type	Description	Example
Path Variables	Used to pass resource identifiers in the URL	/api/loan/status/{loanId}

Request Parameters	Used for filters, pagination, and queries	/api/items?available=true&page=1
Request Body	Complex objects sent in JSON for POST/PUT	@RequestBody LoanFormDTO
DTOs	Used to wrap and validate input data	GrievanceDTO, LoanFormDTO
Service Parameters	Method-level params passed between layers	applyLoan(farmerId, schemeId, amount)
Header Parameters	JWT tokens and authentication data passed securely	Authorization: Bearer <token>

```

@PostMapping("/login")  @ Priyanshu
public ResponseEntity<JwtAuthResponse> createToken(@RequestBody JwtAuthRequest request){
    this.authenticate(request.getUsername(), request.getPassword());
    UserDetails userDetails = userDetailsService.loadUserByUsername(request.getUsername());
    String token = jwtTokenHelper.generateToken(userDetails);

    User user = this.userDao.findByEmail(userDetails.getUsername()).orElseThrow(() -> new ResourceNotFoundException("User", "

    JwtAuthResponse jwtAuthResponse = new JwtAuthResponse();
    jwtAuthResponse.setAccessToken(token);
    jwtAuthResponse.setUser(this.modelMapper.map(user, UserDTO.class));
    return new ResponseEntity<>(jwtAuthResponse, HttpStatus.OK);
}

private void authenticate(String username, String password) { 1 usage  @ Priyanshu
    UsernamePasswordAuthenticationToken authenticationToken = new UsernamePasswordAuthenticationToken(username, password);
    try {
        this.authenticationManager.authenticate(authenticationToken);
    } catch (DisabledException e) {
        throw new DisabledException("User is disabled");
    }
}

```

Figure 39 Example of Parameter Passing / calling

4.6. Validation Checks

Validation checks play a vital role in making sure the data entered and processed in FarmEasy is accurate, secure, and consistent. Performing thorough validation both on the frontend and backend helps maintain the system's reliability and prevents problems that could arise from incorrect data. Since the application manages important processes such as loan approvals, scheme eligibility, grievances, and item bookings, strong validation is necessary to uphold overall system integrity.

Table 8 Examples and Purpose of the Validation checks

Validation Type	Example in FarmEasy	Purpose
-----------------	---------------------	---------

Field-Level Validation	Aadhaar must be 12 digits, amount must be ≥ 1000	Ensures proper input formatting
DTO Validation	@NotNull, @Pattern, @Min, @Size in request DTOs	Validates incoming request payloads
Duplicate Validation	Prevent same grievance or loan from being submitted twice	Avoids redundancy
Role-Based Validation	Only farmers can apply for loans or book items	Ensures action is done by right role

```

public class LoanFormDTO {

    @NotNull(message = "ID is auto-generated and should not be provided")
    private int id;

    ~
    @NotBlank(message = "Name is required")
    @Size(max = 100, message = "Name must not exceed 100 characters")
    private String name;

    ~
    @NotBlank(message = "Email is required")
    @Email(message = "Enter a valid email address")
    private String email;

    ~
    @NotBlank(message = "Contact number is required")
    @Pattern(regexp = "^[6-9]\\d{9}$", message = "Enter a valid 10-digit contact number")
    private String contact;

    @NotBlank(message = "Gender is required")
    private String gender;

    ~
    @Min(value = 18, message = "Age must be at least 18")
    @Max(value = 100, message = "Age must be less than or equal to 100")
    private int age;

    ~
    @Min(value = 300, message = "CIBIL score must be at least 300")
    @Max(value = 900, message = "CIBIL score must not exceed 900")
    private int cibil;

    @NotBlank(message = "Address is required")
    private String address;
}

```

Figure 40 Example of Validation checks

Testing

5.1 Testing Techniques and Testing Strategies Used

1. **Units of Testing:** This means testing different parts of the system, like classes or methods, separately. Usually, mocks or stubs are used in place of external dependencies like DAOs or services. For example, Mockito can mimic these dependencies while testing controller or service classes. Here, the primary goal is to verify that the business logic is sound, that methods produce the desired results, that exceptions are raised when they ought to be, and that side effects are managed appropriately.
2. **Integration Testing:** This type of testing looks at how different components work together — for instance, the service layer interacting with the database. Instead of mocks, real or in-memory databases like H2 are often used. The goal is to verify that components integrate smoothly, data is persisted correctly, and transactions behave as expected.
3. **Functional and end-to-end testing:** Functional tests encompass the entire application flow much as a real user would, often involving sending HTTP queries to the system. Postman, Selenium, REST-assured, and Spring Boot's `@SpringBootTest` are often used technologies with a web environment. These tests ensure that real-world user scenarios and UI/backend interactions work as intended.
4. **Mocking:** This method is about using fake objects in place of the real ones that the code depends on. These fakes behave kinda like the real thing. Mockito is one of the widely used mocking libraries in Java. By using mocks, the code gets tested without actually depending on outside systems — which makes unit testing faster and usually more reliable too.
5. **Testing using Parameters:** This means running the same test again and again but with different input data each time. It helps to check whether the system behaves correctly when used with a range of inputs. Sometimes, minor issues only show up when unusual or edge-case values are passed, so this type of testing really helps catch those.
6. **Behaviour Driven Development (BDD):** BDD follows a way of writing tests that's close to how we speak or write normally. It usually sticks to the “Given–When–Then” format. With tools like Cucumber or JBehave, it becomes easier to create such tests,

and they're much more readable — especially for non-developer people, since it's written more like how business requirements are usually talked about.

5.2. Testing Plan Used

5.2.1. Testing Types used

Table 9 Testing Types Used

Testing Type	Purpose	Tools Used
Unit Testing	Test individual methods in isolation	JUnit, Mockito
Integration Testing	Verify service-repository and business logic integration	JUnit, SpringBootTest, H2
System Testing	End-to-end flow testing	Postman, Swagger
Functional Testing	Validate actual use cases & requirements	Postman
Security Testing	Verify JWT auth, role access control	JUnit, Manual testing
Negative Testing	Check invalid inputs, nulls, unauthorized access	JUnit, Postman
Performance Testing	Ensure the app handles large inputs or load well	JMeter (optional)

5.2.2. Testing Levels & Activities

Table 10 Testing Levels

Level	Activity
Unit	Test each service method (mocking DAOs, verifying outputs & exceptions)
Integration	Test service + repository layer with real H2 DB (SpringBootTest)
System	Test full flow: user applies → booking → payment → grievance
Acceptance	Check if project meets final expected behavior from a user's view

5.3. Testing Report for the Unit test cases and System Test cases

5.3.1. Summary of Testing

Table 11 Test Summary

Testing Type	Total Test Cases	Passed	Failed	Blocked	Coverage
Unit Testing	58	58	0	0	~90%
System Testing	25	25	0	0	100% Functional

5.3.2. Unit Test cases

Table 12 Unit Test cases coverage

Module	No. of Unit Tests	Status	Coverage (%)
User Registration/Login	10	All Passed	95%
Loan Application Service	8	All Passed	90%
Scheme Eligibility (Drools)	5	All Passed	85%
Grievance Handling	6	All Passed	90%
Item Booking Service	7	All Passed	95%
Payment Integration	9	All Passed	88%
Admin/Merchant Services	5	All Passed	90%
Utility Functions (JWT, Validators)	8	All Passed	100%

5.3.3. System Test Cases

Table 13 System Test cases Passed Report

Test Case ID	Description	Expected Result	Status
SYS-01	Register a new farmer	201 Created	Pass
SYS-02	Login with valid credentials	200 OK + JWT token	Pass
SYS-03	Apply for loan with valid inputs	Loan application created	Pass
SYS-04	Check scheme eligibility (CIBIL, Age, Salary)	Eligibility result shown	Pass
SYS-05	Book an item (tractor)	Booking recorded as PENDING	Pass
SYS-06	Make payment using Razorpay	Order created & stored	Pass
SYS-07	Admin approves loan	Status updated to APPROVED	Pass
SYS-08	Raise a grievance	Grievance saved	Pass
SYS-09	Access restricted route without token	403 Forbidden	Pass
SYS-10	Merchant adds new item	Item saved	Pass
SYS-11	Book more than available quantity	400 Bad Request handled well	Pass
SYS-12	View booking/payment history	Accurate data retrieved	Pass
SYS-13	Update payment status post-confirmation	Status updated to SUCCESS	Pass

5.3.4 Bugs Report

All previously identified bugs have been resolved.

5.4. Debugging and Code Improvement

Debugging was really important throughout FarmEasy's development and testing phases. We followed a simple, step-by-step method to quickly find and fix bugs. Tools like SLF4J and Logback helped log incoming requests and errors, making it easier to trace

problems. We also used breakpoints in IntelliJ IDEA frequently to observe how the app behaved in real-time, which helped catch logical errors and unexpected issues.

Whenever exceptions happened during runtime, we carefully examined stack traces to find their exact causes across different layers like controllers, services, and repositories. For testing APIs, tools like Postman and Swagger were really helpful in simulating real user scenarios and edge cases, ensuring every endpoint could handle unexpected inputs smoothly. The H2 database console was a useful tool that let us monitor data changes live while the app was running.

To keep unhandled errors from affecting the user experience, we added well-structured try-catch blocks in key parts of the code. These helped catch exceptions properly and send meaningful error messages back to the frontend. Overall, these debugging steps made the app more reliable and resilient.

This separation of concerns across controller, service, and repository layers helped keep the architecture clean and scalable. Code reusability was increased by extracting repeated validation checks, response formatting, and exception handling into utility classes and common helper methods.

Validation mechanisms were significantly strengthened by using Java Bean Validation annotations like `@Valid`, `@NotNull`, and `@Size` in DTOs. Extra server-side checks were implemented to manage edge cases, like invalid Aadhaar numbers or CIBIL scores beyond the allowed limits. An exception handling system was centralized using Spring's `@ControllerAdvice`, resulting in more consistent and manageable error messages throughout the application.

Security represented another aspect that needed enhancement. The authentication process utilizing JWT was improved to confirm that tokens were accurately checked with filters, and role-based access control was implemented through annotations such as `@PreAuthorize`. The configuration of the Drools rule engine was further enhanced. Rules were divided into several `.drl` files based on schemes and banks, and annotations were included to enhance their clarity and manageability.

Regarding performance enhancement, unnecessary database queries were reduced by employing cached lookups when suitable, and intricate condition checks or nested loops were streamlined. Tools for code quality, such as SonarLint, were incorporated into the

development process to identify code smells and security vulnerabilities at an early stage. Naming conventions were adhered to consistently, and attempts were made to lower cyclomatic complexity in essential modules.

Substantial restructuring was conducted in the payment module. Earlier, Razorpay logic was closely integrated within the controller, complicating testing and maintenance.

This logic was extracted into a dedicated service class, resulting in cleaner and more maintainable code. The use of separate MyOrder and Payment entities helped clearly distinguish between the order generation and final payment confirmation stages.

These debugging and code improvement efforts resulted in faster endpoint response times—reduced by up to 30% in some cases—along with enhanced system security, better maintainability, and easier future updates. Overall, the FarmEasy application became more stable, efficient, and user-friendly as a result of these enhancements.

Table 14 Before vs After Code Snapshot

Criteria	Before	After (Improved)
Payment logic	Razorpay logic tightly coupled in controller	Extracted to PaymentService
Validation	Manual checks in controller	Used @Valid with custom validators
Exception handling	Multiple try-catch in every service	Centralized via @ControllerAdvice
Item booking logic	Hardcoded status assignment	Used enums and default logic at entity level
Loan eligibility logic	Inline Java conditions	Moved to Drools rules for flexibility

System Security Measures

6.1. Database/ Data Security Implementation

FarmEasy relies on MySQL as its main relational database to keep all essential data, including user profiles, loan applications, schemes, payments, and grievances. The database structure enforces strong data integrity through carefully designed entity relationships and constraints. For security, sensitive details like passwords are hashed securely using BCrypt before they're saved, while user authentication uses JWT tokens to maintain secure, stateless sessions. To prevent unauthorized access, data permissions is set according to user roles like administrator, bank, or farmer. The system also use parameterized queries and input validations to avoid threats such as SQL injection. Encrypted HTTPS connections are used to protect all data sent over the network. To ensure data reliability and help monitoring changes for accountability, regular backups and audit logs are also maintained.

```
@Component 5 usages  Priyanshu
public class JwtTokenHelper {

    public static final long JWT_TOKEN_VALIDITY = 5 * 60 * 60; 1 usage
    private static final String SECRET = "afafasfafafasfasfasafacasdasfasxASFACASDFACASDFASFSFADFASFASDAADSCSDFADCVSGCFVADXCcadway
    SecretKey key= Keys.hmacShaKeyFor(Decoders.BASE64.decode(SECRET)); 2 usages

    > public String getUsernameFromToken(String token){ return getClaimFromToken(token, Claims::getSubject); }
    > public Date getExpirationDateFromToken(String token){ return getClaimFromToken(token, Claims::getExpiration); }

    public <T> T getClaimFromToken(String token, Function<Claims, T> claimsResolver) { 2 usages  Priyanshu
        final Claims claims = getAllClaimsFromToken(token);
        return claimsResolver.apply(claims);
    }

    private Claims getAllClaimsFromToken(String token) { 1 usage  Priyanshu
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload();
    }

    private Boolean isTokenExpired(String token) { 1 usage  Priyanshu
        final Date expiration = getExpirationDateFromToken(token);
        return expiration.before(new Date());
    }

    public String generateToken(UserDetails userDetails) { 1 usage  Priyanshu
        Map<String, Object> claims = new HashMap<>();
        return doGenerateToken(claims, userDetails.getUsername());
    }
}
```

Figure 41 JWT Security Implementation

```

spring:
  application:
    name: FarmeasyPortal

  datasource:
    url: jdbc:mariadb://localhost:3308/farmeasy-app?createDatabaseIfNotExist=true
    username: root
    password: mariadb12345
    driver-class-name: org.mariadb.jdbc.Driver
    # Uncomment the following line if needed
    # jpa:
    #   hibernate:
    #     ddl-auto: update

  servlet:
    multipart:
      max-file-size: 20MB
      max-request-size: 20MB

  web:
    resources:
      static-locations: classpath:/static/,classpath:/templates/farmer/

  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true
    validate-on-migrate: true

```

Figure 42 DataBase Implementation

6.2. Creation of User Profiles

```

@PostMapping(("/register/farmer"))  @ Priyanshu
public ResponseEntity<?> registerFarmer(@RequestBody @Valid FarmerDTO farmerDTO) {
    if (userService.isUserExistByEmail(farmerDTO.getEmail())) {
        return new ResponseEntity<>(new ApiResponse("User already exists"), HttpStatus.CONFLICT);
    }

    FarmerDTO farmerRegistered = farmerService.saveFarmer(farmerDTO);
    return new ResponseEntity<>(farmerRegistered, HttpStatus.CREATED);
}

@PostMapping(("/register/bank"))  @ Priyanshu
public ResponseEntity<?> registerBank(@RequestBody @Valid BankDTO bankDTO) {
    if (this.bankService.isBankExistByEmail(bankDTO.getEmail())) {
        return new ResponseEntity<>(new ApiResponse("Bank already exists !!"), HttpStatus.CONFLICT);
    }

    BankDTO bank = this.bankService.addBank(bankDTO);
    return new ResponseEntity<>(new ApiResponse("Bank registered successfully"), HttpStatus.CREATED);
}

```

Figure 43 Code for the creation of user Profiles

6.3. Managing the User Rights

```

107 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
108     http.csrf(AbstractHttpConfigurer::disable);
109
110     http.authorizeHttpRequests(request -> {
111         request.requestMatchers("/api/v1/gov/**").hasRole("GOV");
112         request.requestMatchers(HttpMethod.GET).permitAll();
113         request.requestMatchers("/api/v1/bank/**", "/api/rules/create",
114             "/api/v1/farmer/getFormDetail", "/api/v1/farmer/download").hasRole("BANK");
115         request.requestMatchers("/api/v1/farmer/**", "/api/v1/payments/**").hasRole("FARMER");
116         request.requestMatchers("/api/v1/merchant").hasRole("MERCHANT");
117         request.requestMatchers("/**").permitAll();
118         request.requestMatchers("/v3/api-docs").permitAll();
119         request.requestMatchers(PUBLIC_URLS).permitAll();
120         request.anyRequest().authenticated();
121     })

```

Figure 44 Code for Managing User Rights

Cost Estimation and its Model

The FarmEasy project's development effort and timeline was mostly predicted through code estimation using the Function Point Analysis (FPA) method combined with Expert Judgment. FPA helped in estimating the software size by looking at functional requirements such as inputs (like loan applications and user registration), outputs (like notifications and reports), inquiries (like status checks), internal files (such as user, loan, and scheme databases), and external interfaces (like government and bank systems). The project's modules included user management, loan processing, scheme management, item booking, payment integration, and grievance handling.

The complexity of each module was determined by calculating function points and assigning weights based on the number and complexity of transactions and data components involved. This model allowed the team to estimate the total lines of code (LOC) roughly, as well as development time, by mapping function points to historical productivity rates. Expert judgment by senior developers supplemented this estimation by considering project-specific factors like use of frameworks (Spring Boot, Drools) and security layers (JWT), which influenced coding and testing effort.

Using this combined estimation approach ensured a realistic schedule, resource allocation, and helped track progress effectively throughout the development lifecycle.

Table 15 Cost Estimation

Cost Item	Estimated Monthly Cost (INR)
Free-tier Cloud Hosting	₹0
Domain (Optional)	₹100
SSL Certificate	₹0
Maintenance & Support	₹0
Payment Gateway Charges	Variable (pay-per-use)
Total Estimated Cost	~₹100 - ₹200

Future Scope

FarmEasy has lots of room to expand and improve so it can serve farmers, banks, and other stakeholders better. Future versions might add AI-powered loan eligibility analysis to make approvals faster and more accurate. Mobile app support could also benefit farmers who have slow or unreliable internet connections. To assist farmers in making better judgments, the idea might be expanded to incorporate crop guidance and real-time meteorological data. People from different geographical locations will find the system easier to use if it supports multiple languages. Additionally, developing advanced analytics dashboards for government agencies and institutions may help them better understand user behavior and scheme performance. By establishing connections with more lending institutions and payment options, financial inclusion could be further enhanced. Finally, incorporating blockchain-based data security can improve loan and transaction records' dependability and transparency.

Bibliography

- Spring Framework Documentation (<https://spring.io/projects/spring-boot>)
- ModelMapper Documentation (<https://modelmapper.org/getting-started/>)
- Drools Rule Engine Documentation
(https://docs.jboss.org/drools/release/latest/drools-docs/html_single/)
- JWT (JSON Web Token) Introduction and Standards (<https://jwt.io/introduction/>)
- BCrypt Password Hashing (<https://www.baeldung.com/spring-security-registration-password-encoding-bcrypt>)
- MySQL Documentation (<https://dev.mysql.com/doc/>)
- Postman API Testing Tool (<https://www.postman.com/>)
- SonarLint for Code Quality (<https://www.sonarlint.org/>)
- Agile Software Development Principles - Beck, Kent, et al. Manifesto for Agile Software Development. 2001.

Data Dictionary

Table 16 Data Dictionary for the constraints

Entity/Table	Field Name	Data Type	Description
User	id	String	Unique identifier for the user
	email	String	User's email address (login username)
	password	String	Encrypted user password
	role	String	Role of the user (e.g., BANK, FARMER)
	createdAt	Timestamp	Account creation timestamp
Bank	id	String	Unique identifier for the bank
	bankName	String	Official name of the bank
	email	String	Bank's contact email
	password	String	Hashed password for bank user
	address	String	Bank's address
Scheme	id	Integer	Unique scheme ID
	bankId	String	Foreign key linking to Bank
	schemeName	String	Name of the scheme
	schemeCode	String	Code identifier for the scheme
	description	String	Description of scheme benefits and rules
	eligibility	String	Eligibility criteria description
	min_salary	Double	Minimum salary requirement
	max_salary	Double	Maximum salary limit
	cibil_score	Integer	Minimum CIBIL credit score required
	roi	Double	Rate of interest applicable

	tenure	Integer	Tenure duration in months
Apply	id	Integer	Unique application ID
	farmerId	String	Foreign key linking to Farmer
	bankId	String	Foreign key linking to Bank
	schemeId	Integer	Foreign key linking to Scheme
	status	Enum	Current application status (PENDING, APPROVED, etc.)
	review	String	Review or comments on the application
	statusDate	Date	Date when the status was last updated
Farmer	id	String	Unique identifier for the farmer
	name	String	Farmer's full name
	email	String	Farmer's email address
	phoneNumber	String	Contact phone number
	address	String	Residential address
Grievances	id	Integer	Unique grievance ID
	farmerId	String	Foreign key linking to Farmer
	bankId	String	Foreign key linking to Bank
	message	String	Details of the grievance
	status	Enum	Current status of grievance (OPEN, CLOSED)